

Document number	P3216R1
Date	2025-11-06
Audience	LEWG, SG9 (Ranges)
Reply-to	Hewill Kang <hewillk@gmail.com>

views::slice

- - Abstract
 - Revision history
 - Discussion
 - Design
 - Implementation experience
 - Proposed change
 - References

Abstract

This paper proposes the Tier 1 adaptor `views::slice` (as described in [P2760](#)) to enhance the C++29 ranges library. Notably, this is the first standard range adaptor that accepts two arguments — `start` and `end` — to specify the interval `[start, end)` for slicing a range.

Revision history

R0

Initial revision.

R1

Introduce new `slice_view` class based on feedback from SG9 St. Louis.

R2

Apply specialization for `optional` based on feedback from SG9 Kona.

Discussion

Slicing — a means of extracting a contiguous subrange from a sequence by specifying a start and end index — is a fundamental operation in modern programming. Many mainstream languages, such as Python and Rust, offer built-in slice syntax, making it a familiar and expected feature for developers. In the C++ ecosystem, while the Ranges library has greatly enhanced composability and expressiveness, it currently lacks a direct, ergonomic, and standard way to perform slicing by index.

Why `views::slice` is needed

- Familiarity

Developers often need to access a specific segment of a range, whether for data processing, algorithmic partitioning, or windowed computations. Introducing `views::slice` aligns C++ with the expectations set by other languages, reducing the cognitive gap for new and experienced programmers alike.

- Readability

While it is possible to achieve slicing today with `views::drop(start) | views::take(end - start)`, this composition, while valid, may obscure the programmer's intent due to its verbosity. A single `views::slice(start, end)` call is both concise and self-documenting, improving code clarity and maintainability.

- Generic and Robust Semantics

Unlike `subrange` and `counted`, which have limitations when working with only-input or non-sized range types, `views::slice` can be designed to work generically with any range, making it broadly applicable. Its semantics — returning the interval `[start, end)` and gracefully handling out-of-bounds indices — are both predictable and robust.

Given the above, the introduction of `views::slice` fills a clear gap in the C++ Ranges library, providing a direct, expressive, and interoperable way to extract subranges by index. It aligns C++ with industry standards, improves code readability, and empowers developers to write more concise and correct range-based code. For these reasons, `views::slice` is a valuable and timely addition to C++29:

```
string_view text = "Hello, world!";
auto sub1 = text | views::slice(7, 12); // "world"

vector v = {1, 2, 3, 4, 5};
auto sub3 = v | views::slice(1, 10);    // [2, 3, 4, 5]
auto sub2 = v | views::slice(2, 2);    // empty range
auto sub4 = v | views::slice(5, 10);    // empty range
```

Design

The second argument should be *end* instead of *size*

An alternative design for `slice` is to accept a starting index and a *size*. However, the author prefers to use the end index as the second parameter as this is intuitive and consistent with other language syntaxes:

Language	Syntax	Stride Support	Negative Indices	Out-of-Bounds Behavior	Notes
Python	<code>a[start:end:step]</code>	✅ Yes	✅ Yes	✅ Truncated	
Rust	<code>&a[start..end]</code>	❌ No	❌ No	❌ Panics	Need to use <code>.iter().step_by(n)</code> to stride
Go	<code>a[start:end]</code>	❌ No	❌ No	❌ Panics	<code>a[start:end:max]</code> controls capacity, not stride
JavaScript	<code>a.slice(start, end)</code>	❌ No	✅ Yes	✅ Truncated	
Ruby	<code>a[start, length]</code> or <code>a[start..end]</code>	❌ No	✅ Yes	✅ Truncated	

This is the de facto standard for slicing.

Motivation for a dedicated `slice_view` class

As stated in [P2214](#): "`slice(M, N)` is equivalent to `views::drop(M) | views::take(N - M)`, and you couldn't do much better as a first class view. range-v3 also supports a flavor that works as `views::slice(M, end - N)` for a special variable `end`, which likewise be equivalent to `r | views::drop(M) | views::drop_last(N)`."

While `views::drop` and `views::take` provide compositional power, a dedicated `slice_view` offers better API consistency, performance, and expressiveness. It enables more robust and efficient handling of subranges for the following reasons:

- `base()` member Consistency

When composing `views::drop(M) | views::take(N - M)`, the resulting view is a `take_view`. Calling `base()` on this returns the underlying `drop_view` rather than the original range. This breaks the expectation that `base()` should always provide access to the original range, making code that needs to retrieve the true base more indirect as `.base().base()`.

- Global Range Awareness and `reserve_hint()`

A dedicated `slice_view` holds both the start and end indices, allowing it to globally understand the intended subrange. This enables it to unconditionally provide a `reserve_hint()` member that returns `end - start`, accurately reflecting the

expected size of the slice in most cases. With a drop-take composition, this global knowledge is lost, making it difficult or impossible to provide such optimizations. For example, `drop_view` only provides `reserve_hint()` if the underlying range models `approximately_sized_range`.

- Performance Overhead

Indirect composition through multiple views (i.e., layering `drop_view` and `take_view`) can introduce additional iterator wrappers and indirections. This may result in reduced performance due to extra function calls and less efficient inlining or optimization opportunities compared to a single, purpose-built `slice_view`.

- Unified Boundary Handling

A dedicated `slice_view` can perform comprehensive boundary checks and handle out-of-range conditions in a single place, ensuring predictable and robust behavior. In contrast, drop-take composition splits responsibility, potentially leading to inconsistent or redundant checks.

- Better Debug-ability

A dedicated `slice_view` results in a named, standalone view type that directly reflects the user's intent. In contrast, composing `drop_view` and `take_view` layers results in a nested and opaque type, which can be harder to inspect or reason about in debugging sessions.

- Better Future Extensibility

`slice_view` makes it easier to add new features or enhancements in the future. Any extensions or improvements can be implemented directly in `slice_view` without needing to consider the internal logic or compatibility of both `drop_view` and `take_view`. This simplifies maintenance and enables more flexible evolution of the slicing functionality.

Handling of out-of-bounds

It should be noted that in `range/v3`, `views::slice` is also implemented with a dedicated view class, but it does **not** perform any boundary checking. It always assumes that the provided `start` and `end` indices are within valid bounds of the underlying range:

```
auto ints = {1, 2, 3, 4, 5};
auto slice = ints | ranges::v3::views::slice(3, 9);
std::println("{} ", slice); // prints [4, 5, 0, 0, 0, 2147483647]
```

This design makes its `views::slice` effectively an unchecked version of slicing. If the indices are out of range, the behavior is undefined, potentially leading to runtime errors or undefined behavior. From a naming perspective, the `range/v3` version would be more accurately described as `views::unchecked_slice` or `views::slice_exactly`, reflecting its lack of safety checks.

In contrast, the proposed `views::slice` includes comprehensive boundary checking just like `views::take` and `views::drop`; it will safely adjust or clamp the specified indices as appropriate, ensuring well-defined and predictable behavior.

Special variable `end` is not support

In `range-v3`, the special variable `end` is supported in `views::slice`, allowing users to write expressions like `views::slice(M, end - N)` to indicate slicing from index `M` up to `N` elements before the end of the range.

While this can be expressive in certain scenarios, the author believes it is unnecessary and potentially problematic for several reasons.

First, introducing a special variable such as `end` can make the syntax less clear and more confusing, especially for users who expect a straightforward two-index slicing interface similar to what is found in other mainstream languages. This added complexity may hinder readability and increase the learning curve for new users.

Second, and more importantly, `range-v3`'s implementation does not perform boundary checking for the end. It assumes that the indices provided are always valid, which is fundamentally different from our proposed design. Supporting end-based expressions in a boundary-checked implementation introduces challenges, particularly for input ranges, since evaluating something like `end - N` would require traversing the entire range, which is infeasible for single-pass input ranges.

In summary, while the `end` variable enables some expressive patterns, it complicates the interface and is incompatible with a robust, boundary-checked design. For these reasons, the author does not adopt this feature in the proposal.

Stride overload is not provided

As described in the table above, Python also allows an optional stride (step) parameter, its slice syntax `[start:end:step]` enables users to select every *n*th element or even reverse the sequence by specifying a negative stride.

However, JavaScript, Go, and many other languages with slicing capabilities (such as Ruby, Swift, or Kotlin) do not include stride as part of their native slice syntax; instead, users must use separate functions or methods to achieve similar effects. Rust's standard slice syntax does not support stride directly; users must use iterators like `.iter().step_by(n)` to achieve striding.

This supports the case against overloading `views::slice` with a stride parameter in C++, which, already provides a clear and composable way to achieve striding via `views::stride`. Chaining adaptors like `views::slice(M, N) | views::stride(P)` makes the intent and order of operations clear, whereas adding a stride overload to `views::slice` could blur the distinction between slicing and stepping, making the API less intuitive.

For these reasons, the author does not support stride.

Specialization for return types

For certain well-known range types such as `iota_view`, `subrange`, or `span`, and so on, `views::slice(start, end)` can be optimized to return a view of the same type with adjusted bounds, rather than wrapping it in a generic `slice_view`. This follows the precedent set by `views::take` and `views::drop`, and can lead to fewer template instantiations, improved compile times, and better performance.

Handling of non-sized range

For ranges that model `sized_range`, the subrange's start and end positions can be computed precisely. In such cases, the implementation can advance the iterator by the specified offset and construct a `counted_iterator` with the exact number of elements in the slice.

For ranges that do not model `sized_range`, the precise end position *cannot* be determined in advance. The implementation must instead use the bound-preserving overload of `ranges::next` to advance the iterator to the starting position. The resulting iterator is then wrapped in a `counted_iterator` with the *desired* count. Since the base range may not contain enough elements, a custom sentinel is used to detect the actual end of the range. This sentinel terminates iteration either when the count is exhausted or when the underlying iterator reaches the end of the base range, similar to the behavior of `take_view::sentinel`.

Conditionally const-iterable

To satisfy the requirement that `begin()` of a view operates in amortized constant time, the implementation must cache the computed starting iterator when it cannot be obtained in constant time. In particular, when the base range does not model `random_access_range` or `sized_range`, advancing the `begin()` iterator to the desired offset may require $O(n)$ time. In such cases, the result of this computation must be stored internally to avoid repeated traversal on subsequent calls to `begin()`, which means that `slice_view` may not be `const`-iterable in all cases.

Implementation experience

The author implemented `views::slice` based on libstdc++, see [here](#).

Proposed change

This wording is relative to [N5014](#).

1. Add a new feature-test macro to 17.3.2 [\[version.syn\]](#):

```
#define __cpp_lib_ranges_slice 2026XXL // freestanding, also in <ranges>
```

2. Modify 25.2 [\[ranges.syn\]](#), Header `<ranges>` synopsis, as indicated:

```
// mostly freestanding
#include <compare>           // see \[compare.syn\]
#include <initializer_list>  // see \[initializer.list.syn\]
#include <iterator>          // see \[iterator.synopsis\]
```

```

namespace std::ranges {
    [...]
    namespace views { inline constexpr unspecified drop_while = unspecified; }

    // [range.slice], slice view
    template<view> class slice view;

    template<class T>
    constexpr bool enable_borrowed_range<slice view<T>> =
        enable_borrowed_range<T>;

    namespace views { inline constexpr unspecified slice = unspecified; }
    [...]
}

```

3. Add **25.7.?** Slice view [range.slice] after 25.7.13 [range.drop.while] as indicated:

-1- A slice view presents elements from index N (inclusive) up to index M (exclusive) of another view, or all elements from N to the end if M exceeds the range, or an empty view if there are fewer than N elements.

-2- The name `views::slice` denotes a range adaptor object ([range.adaptor.object]). Let E , F and G be expressions, let T be `remove_cvref_t<decltype((E))>` and D be `range_difference_t<decltype((E))>`. If `decltype((F))` and `decltype((G))` do not model `convertible_to<D>`, `views::slice(E, F, G)` is ill-formed. Otherwise, the expression `views::slice(E, F, G)` is expression-equivalent to:

(2.1) — If T is a specialization of `empty_view`, `repeat_view`, `span`, `basic_string_view`, or `optional`, or T models `random_access_range` and `sized_range` and is a specialization of `iota_view`, or `subrange`, then $E \mid \text{views::drop}(F) \mid \text{views::take}(\text{static_cast}<D>(G) - \text{static_cast}<D>(F))$, except that F is evaluated only once.

(2.2) — Otherwise, `slice_view(E, F, G)`.

-3- [Example 1]:

```

auto ints = views::iota(0);
auto fifties = ints | views::slice(50, 60);
println("{} ", fifties); // prints [50, 51, 52, 53, 54, 55, 56, 57, 58, 59]

```

— end example]

[25.7.?.2] Class template `slice_view` [range.slice.view]

```

namespace std::ranges {
    template<view V>
    class slice_view : public view_interface<slice_view<V>> {
    private:
        V base_ = V(); // exposition only
        range_difference_t<V> from_ = 0; // exposition only
        range_difference_t<V> to_ = 0; // exposition only

        // [range.slice.sentinel], class slice_view::sentinel
        class sentinel; // exposition only

    public:
        slice_view() requires default_initializable<V> = default;
        constexpr explicit slice_view(V base, range_difference_t<V> from,
                                     range_difference_t<V> to);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr auto begin()
            requires (!simple_view<V> &&
                     random_access_range<const V> && sized_range<const V>);

        constexpr auto begin() const
            requires random_access_range<const V> && sized_range<const V> {
            return ranges::begin(base_) + std::min(from_, ranges::distance(base_));
        }
    }
}

```

```

constexpr auto end()
    requires (!simple-view<V> &&
              random_access_range<const V> && sized_range<const V>)) {
    if constexpr (sized_range<V>) {
        if constexpr (random_access_range<V>)
            return ranges::begin(base_) + std::min(to_, ranges::distance(base_));
        else
            return default_sentinel;
    } else if constexpr (sized_sentinel_for<sentinel_t<V>, iterator_t<V>>)
        return default_sentinel;
    else
        return sentinel(ranges::end(base_));
}

constexpr auto end() const
    requires random_access_range<const V> && sized_range<const V> {
    return ranges::begin(base_) + std::min(to_, ranges::distance(base_));
}

constexpr auto size() requires sized_range<V> {
    auto n = ranges::distance(base_);
    return static_cast<range_size_t<V>>(std::min(to_, n) - std::min(from_, n));
}

constexpr auto size() const requires sized_range<const V> {
    auto n = ranges::distance(base_);
    return static_cast<range_size_t<const V>>(std::min(to_, n) - std::min(from_, n));
}

constexpr auto reserve_hint() {
    if constexpr (approximately_sized_range<V>) {
        auto n = static_cast<range_difference_t<V>>(ranges::reserve_hint(base_));
        return to-unsigned-like(std::min(to_, n) - std::min(from_, n));
    }
    return to-unsigned-like(to_ - from_);
}

constexpr auto reserve_hint() const {
    if constexpr (approximately_sized_range<const V>) {
        auto n = static_cast<range_difference_t<const V>>(ranges::reserve_hint(base_));
        return to-unsigned-like(std::min(to_, n) - std::min(from_, n));
    }
    return to-unsigned-like(to_ - from_);
}
};

template<class R>
slice_view(R&&, range_difference_t<R>, range_difference_t<R>)
    -> slice_view<views::all_t<R>>;
}

constexpr explicit slice_view(V base, range_difference_t<V> from,
                             range_difference_t<V> to);

```

-1- *Preconditions*: `from >= 0` and `from <= to` are each `true`.

-2- *Effects*: Initializes `base_` with `std::move(base)`, `from_` with `from`, and `to_` with `to`.

```

constexpr auto begin()
    requires (!simple-view<V> &&
              random_access_range<const V> && sized_range<const V>));

```

-3- *Returns*:

(3.1) — If `V` models `sized_range`:

(3.1.1) — If `V` models `random_access_range`,

`ranges::begin(base_) + std::min(from_, ranges::distance(base_))`

(3.1.2) — Otherwise, let `n` be `ranges::distance(base_)`,

```
counted_iterator(ranges::next(ranges::begin(base_), std::min(from_, n)),
                 std::min(to_, n) - std::min(from_, n))
```

(3.2) — Otherwise, if `sentinel_t<V>` models `sized_sentinel_for<iterator_t<V>>`, let `it` be `ranges::begin(base_)` and `n` be `ranges::end(base_) - it`,

```
counted_iterator(ranges::next(std::move(it), std::min(from_, n)),
                 std::min(to_, n) - std::min(from_, n))
```

(3.3) — Otherwise, `counted_iterator(ranges::next(ranges::begin(base_), from_, ranges::end(base_)), to_ - from_)`.

-4- *Remarks:* In order to provide the amortized constant-time complexity required by the `range` concept when `slice_view` models `forward_range`, this function caches the result within the `slice_view` for use on subsequent calls.

[*Note 1:* Without this, applying a `reverse_view` over a `slice_view` would have quadratic iteration complexity. — *end note*]

[25.7.?.3] Class `slice_view::sentinel` [range.slice.sentinel]

```
namespace std::ranges {
    template<view V>
    class slice_view<V>::sentinel {
    private:
        sentinel_t<V> end_ = sentinel_t<V>();    // exposition only

    public:
        sentinel() = default;
        constexpr explicit sentinel(sentinel_t<V> end);

        constexpr sentinel_t<V> base() const;

        friend constexpr bool operator==(const counted_iterator<iterator_t<V>>& x, const sentinel& y);
    };

    constexpr explicit sentinel(sentinel_t<V> end);
```

-1- *Effects:* Initializes `end_` with `end`.

```
constexpr sentinel_t<V> base() const;
```

-2- *Effects:* Equivalent to: `return end_;`

```
friend constexpr bool operator==(const counted_iterator<iterator_t<V>>& x, const sentinel& y);
```

-3- *Effects:* Equivalent to: `return x.count() == 0 || x.base() == y.end_;`

References

[P2760R1]

Barry Revzin. A Plan for C++26 Ranges. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r1.html>